

Intro To Database

(Database Fundamental using MySQL)

Made By : Mohamed ELshafei

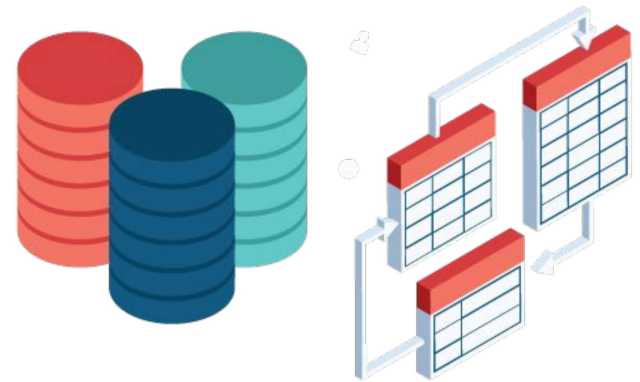
Edited By: Hend Samir

Hend Samir



Agenda

- Data & Information.
- File Based System.
- What is Database, Database System?
- DBMS & its functions.
- Database Properties.
- Advantages and Disadvantages of Database Systems.
- DB Architecture.
- Who Deals with Database.
- Data Models.





Data & Information

- **Data** is the raw input (numbers, characters, images...) which when processed or arranged makes meaningful output (**Information**).
- **Data** is the lowest level of knowledge and **information** is the second level.
- **Data** by itself alone is not significant. **Information** is significant by itself.
- Observations and recordings are done to obtain **data**, while analysis and processing are done to obtain **information**.



File Based System

Early attempt to computerize the manual filing system , **each Program defines and manages its own data.**

- Separation & Isolation Of data (each user has a copy)
- Data Redundancy (Duplication of data)
- No Database integrity
- All programs maintain metadata for each file they use
- Incompatible File Formats
- Non-standard way to query





What is a database?

- A database is an organized collection of related data.”
- The data is typically organized to model relevant aspects of reality in a way that supports processes requiring this information



Database Management System(DBMS)

- It is the intermediate layer between the database and the programs that access the data.
- It is collection of programs that enables users to create and maintain a database.





DBMS Functions

- Constructing Database.
- Manipulating Database.
- Data Security
- Data Integrity.
- Concurrency.
- Backup & Recovery.
- Data Dictionary (Meta Data).
- Performance.





DBMS Other Functions

- Text/Number/Image/Audio/Video.
- Special Data (like GIS).
- Time Series(data stored versus the time).
- Data Mining



Advantages of Database

- Controlling Redundancy.
- Sharing Data.
- Restricting unauthorized Access.
- Enforcing Integrity Constraints.
- Inconsistency can be avoided.
- Providing Backup and Recovery



Disadvantages of Database

- It needs expertise to use.
- DBMS itself is expensive.
- DBMS may be incompatible with any other available DBMS.



Database System

Users



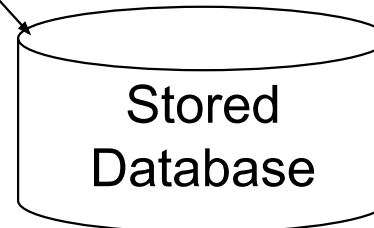
Application Programs



Software to process
Queries

**DBMS
Software**

Software to access Stored Data





Database System

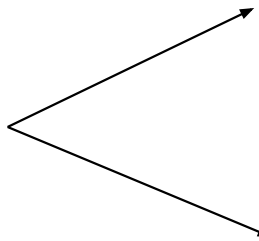
□ A database system is composed of :

- The database.

- The Software.

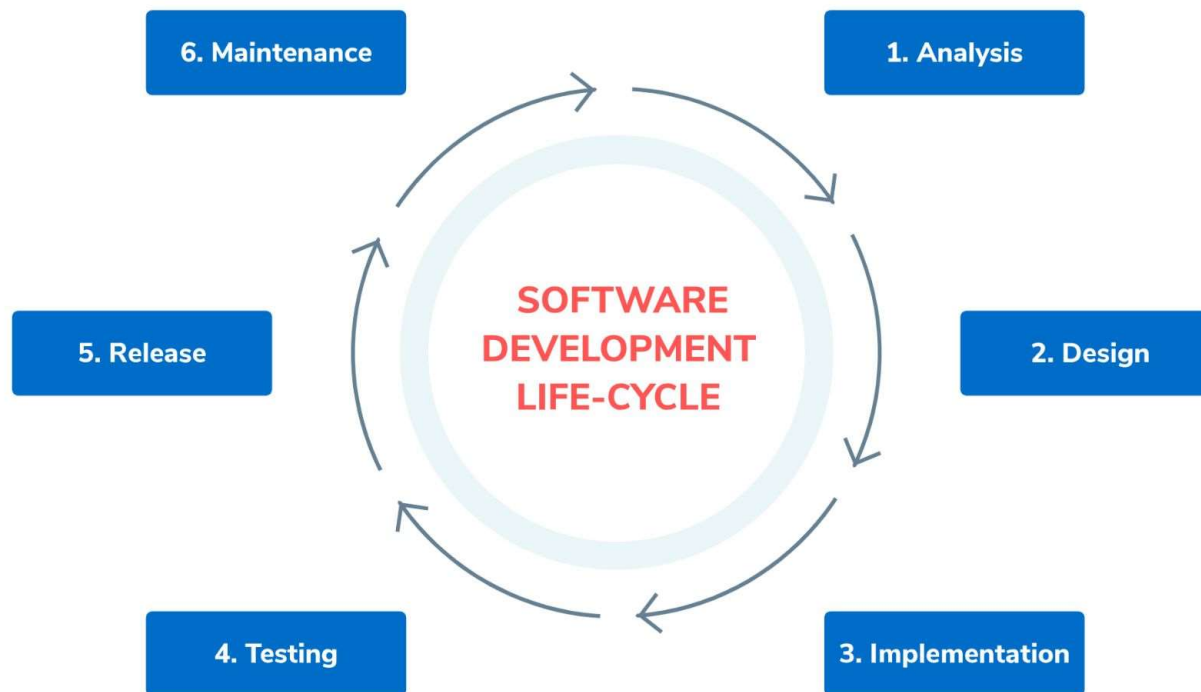
Application Program

DBMS



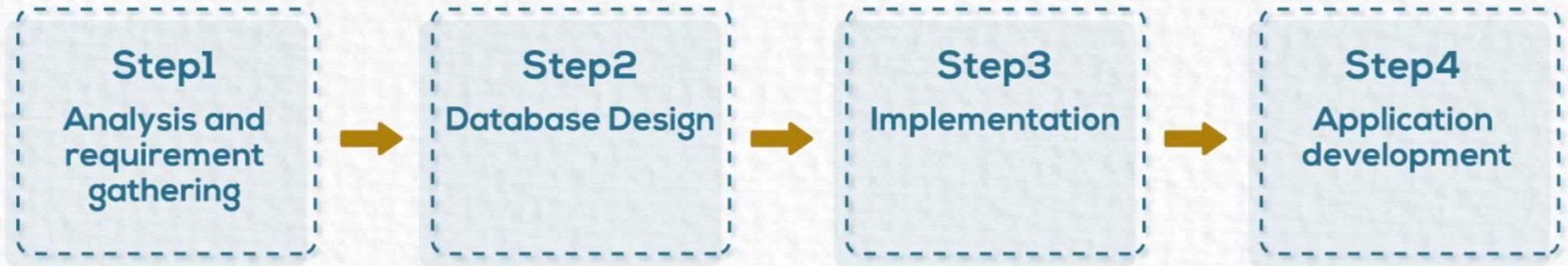


Software development life cycle





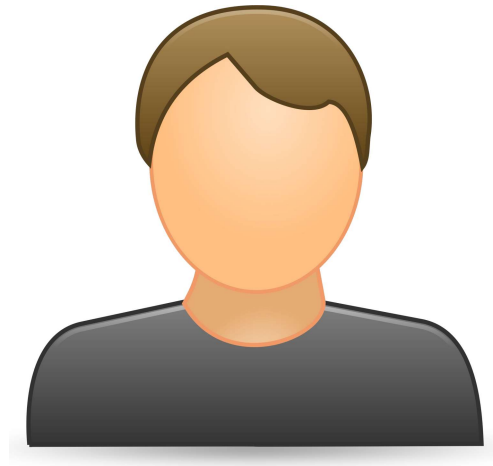
Database development life cycle





Database Users

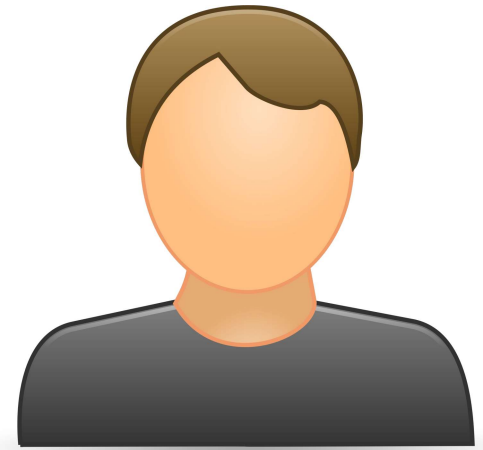
- ❑ System Analysts
 - Business analysis and requirements gathering
- ❑ Database Designer
 - create database design (Conceptual Schema)
- ❑ Database Administrator (DBA)
 - Install DBMS.
 - Create Users and authorize access to db .
 - Maintain DB performance





Database Users Cont.

- ❑ Application programmers
 - Develop , test and debug the application .
- ❑ End users

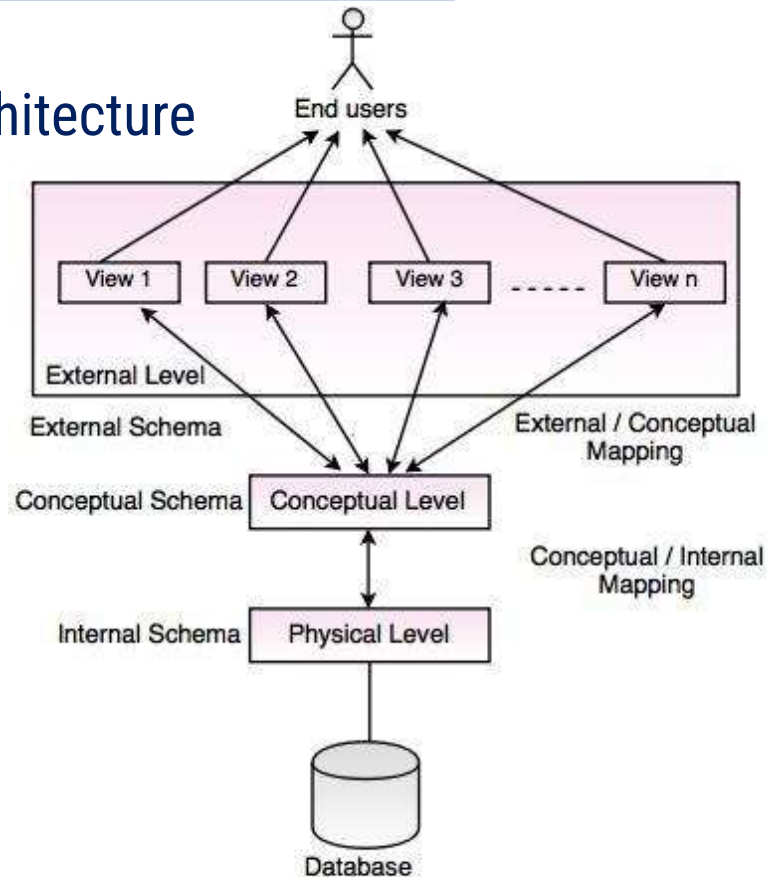




Three Level/Schema Architecture

The three levels of database architecture

1. External Level
2. Conceptual Level
3. Physical Level





External Level

- ☐ External level is related to the data which is viewed by individual end users.
- ☐ This level includes a no. of user views or external schemas.
- ☐ This level is closest to the user.
- ☐ External view describes the segment of the database that is required for a particular user group and hides the rest of the database from that user group.



Conceptual Level

- ☐ Conceptual level describes the structure of the whole database for a group of users.
- ☐ It is also called as the data model.
- ☐ Conceptual schema is a representation of the entire content of the database.
- ☐ These schema contains all the information to build relevant external records.
- ☐ It hides the internal details of physical storage.



Physical Level

- ❑ Physical level describes the physical storage structure of data in database.
- ❑ It is also known as Internal Level.
- ❑ At lowest level, it is stored in the form of bits with the physical addresses on the secondary storage device.
- ❑ At highest level, it can be viewed in the form of files.
- ❑ The internal schema defines the various stored data types. It uses a physical data model.



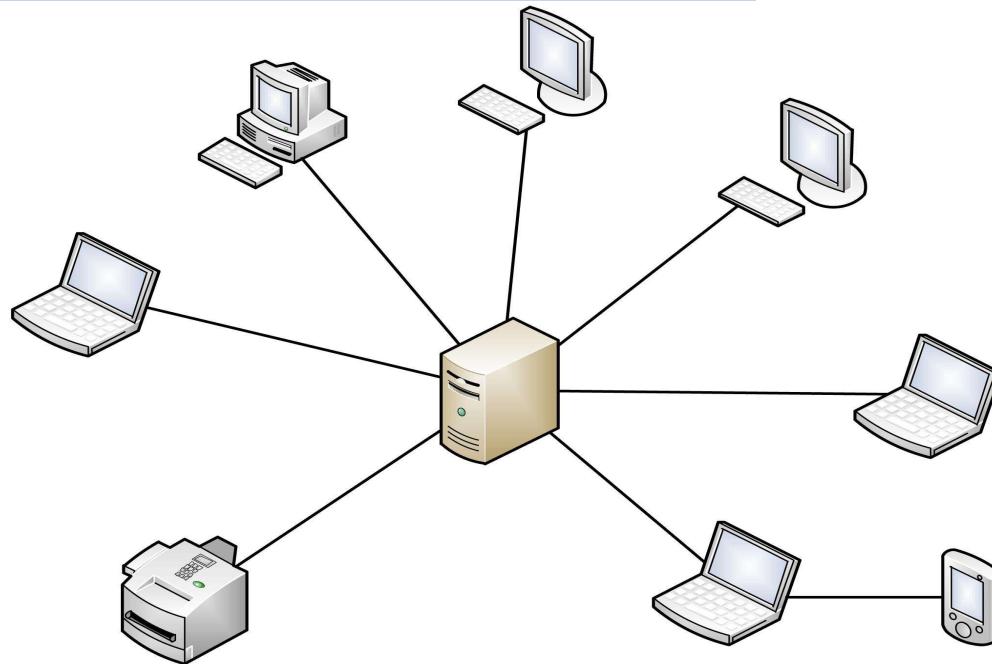
Centralized Database Environment

❑ Mainframe Environment

- ▶ Everything is on one machine and all processing made on one machine
- ▶ The processing depends on server.
- ▶ **Disadvantages**
 - ▶ The processing depends on one server
 - ▶ The performance is very slow
 - ▶ Database and application layer has single point of failure



Mainframe Environment

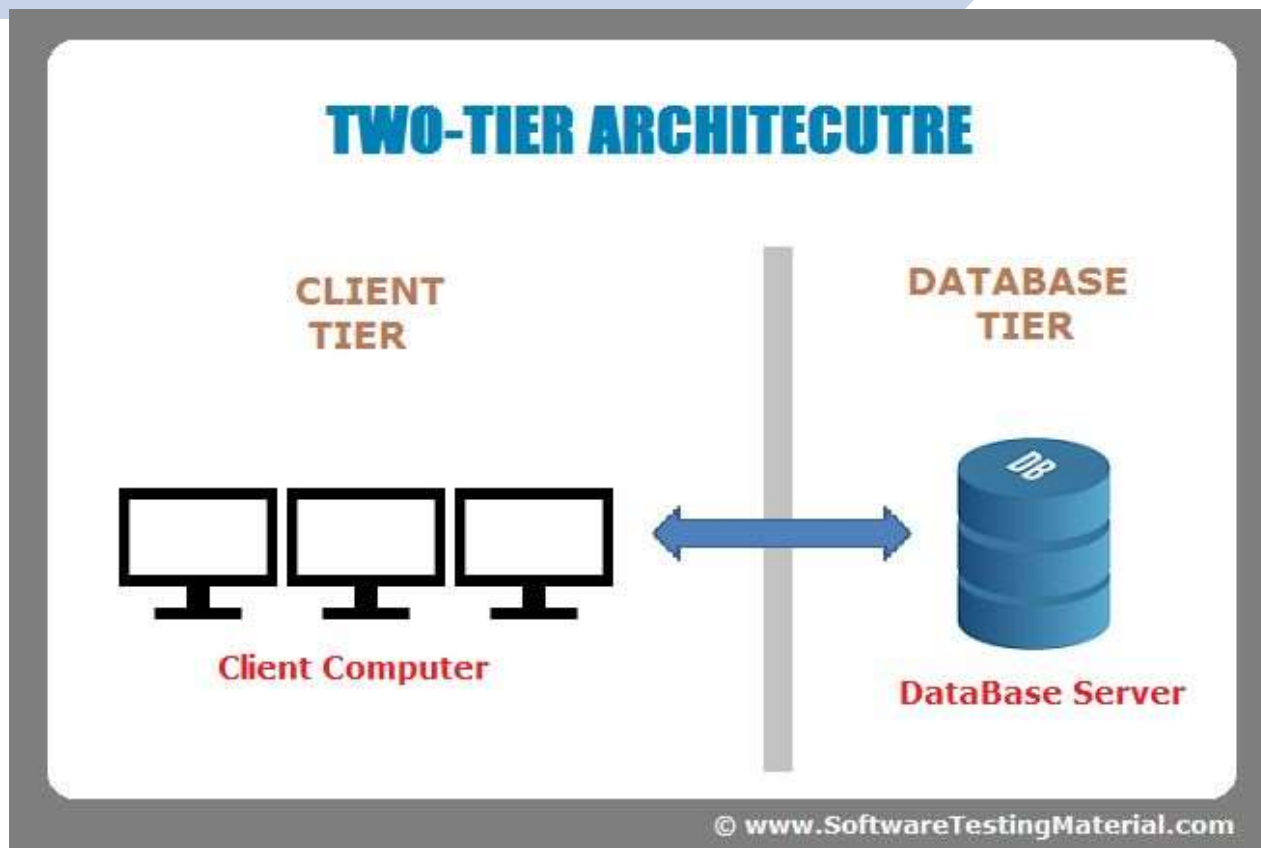


Centralized Database Environment Cont..

Client/Server Environment

- ▶ Two-tier environment
 - ▶ The first tier is a database server .
 - ▶ The second tier is my client. Application is installed locally on every end-user machine.

Client / Server Environment





Client / Server Computing Environment

- **Advantages:**

- ▶ Application layer isn't a single point of failure.

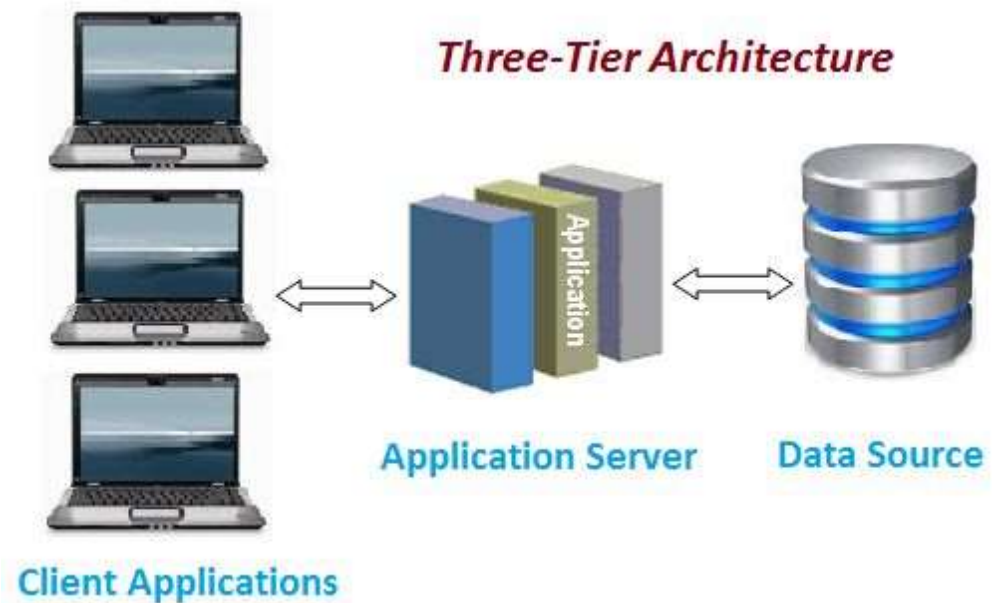
- **Disadvantages**

- ▶ Database is a single point of failure.
 - ▶ High cost for support.
- 

Centralized Database Environment Cont..

Internet Computing environment (Three-tier architecture)

- ▶ Three-tier environment
 - ▶ Thin Client.
 - ▶ Application Server.
 - ▶ Database Server.





Internet Computing Environment

- **Advantages:**

- ▶ lower cost for support and maintenance.

- **Disadvantages**

- ▶ Application server is a single point of failure.
- ▶ Database is a single point of failure.

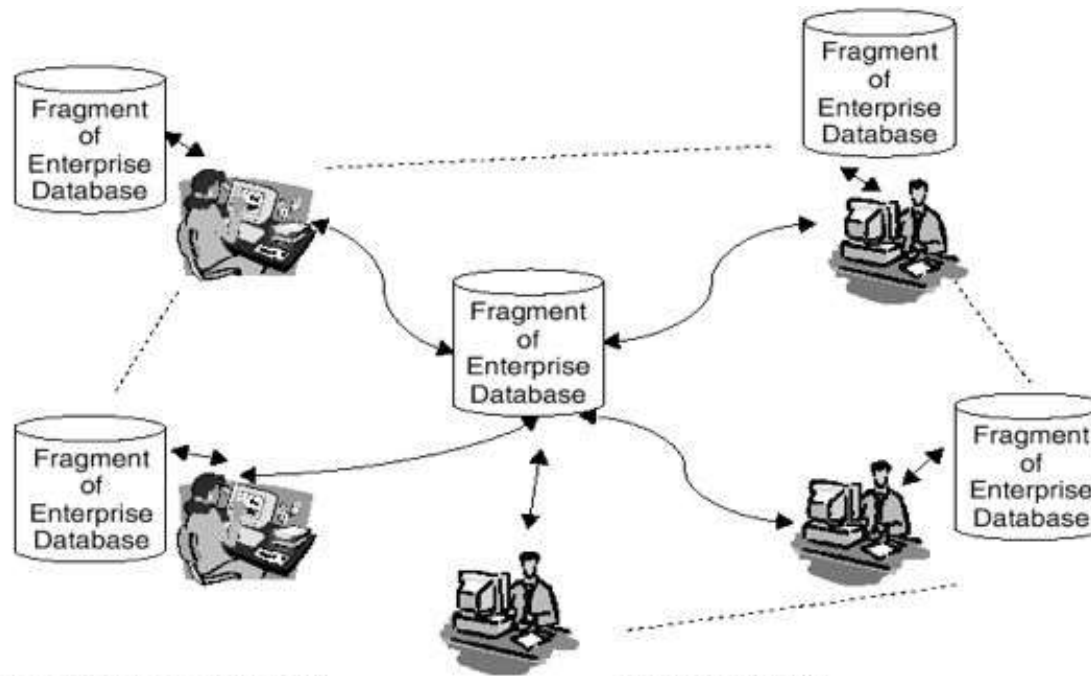


Distributed Database

-  **Replication (Copy - Past)**
 - ▶ **Partial Replication**
 - ▶ **Full Replication**
 -  **Fragmentation (Cut - Past)**
- 



Distributed Database



For global and spread-out organizations, centralized databases not economical. Enterprise data distributed across multiple computer systems.

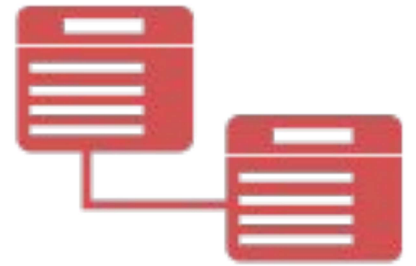
Two categories:

Homogeneous databases
Heterogeneous databases

ER Model

Entity Relationship Diagram

Concepts



ER Model

- **ER Model** is a high-level data model, developed by Chen in 1976. This model defines the data elements and relationships for a specified system. It is useful in developing a conceptual design for the database & is very simple and easy to design logical view of data.



Importance of ER Model

- ERD stands for Entity Relationship diagram.
- ER diagram shows the relationship between objects, places, people, events etc. within that system.
- ER Model is a graphical representation for designing the structure.
- Without ER diagrams you cannot make a database structure & write production code.
- It displays the clear picture of the database structure.

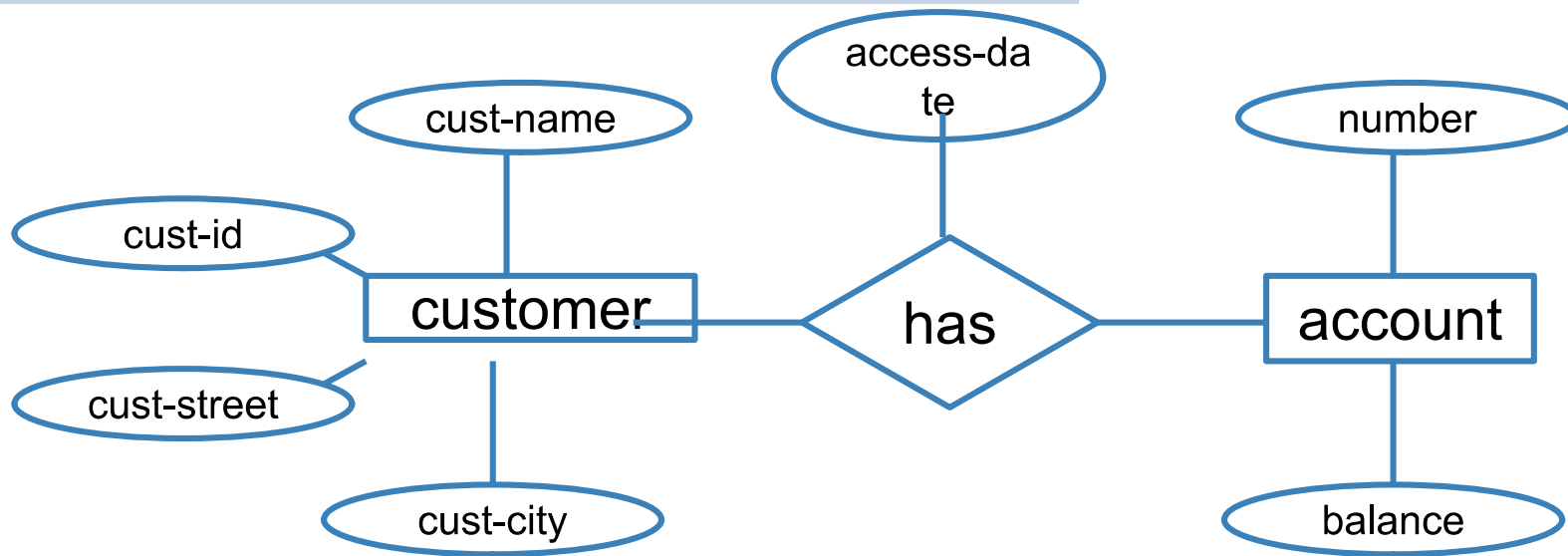


Basic constructs of the E-R model

- **Entities** - person, place, object, event, concept (often corresponds to a real time object that is distinguishable from any other object)
- **Attributes** - property or characteristic of an entity type (often corresponds to a field in a table)
- **Relationships** – link between entities (corresponds to primary key-foreign key equivalencies in related tables)



ER Diagram: Starting Example



- Rectangles: entity sets
- Diamonds: relationship sets
- Ellipses: attributes



Entity

- A definable thing—such as a person, object, concept or event—that can have data stored about it.
- Think of entities as nouns.
- Examples: a customer, student, car or product.
- Typically shown as a rectangle.

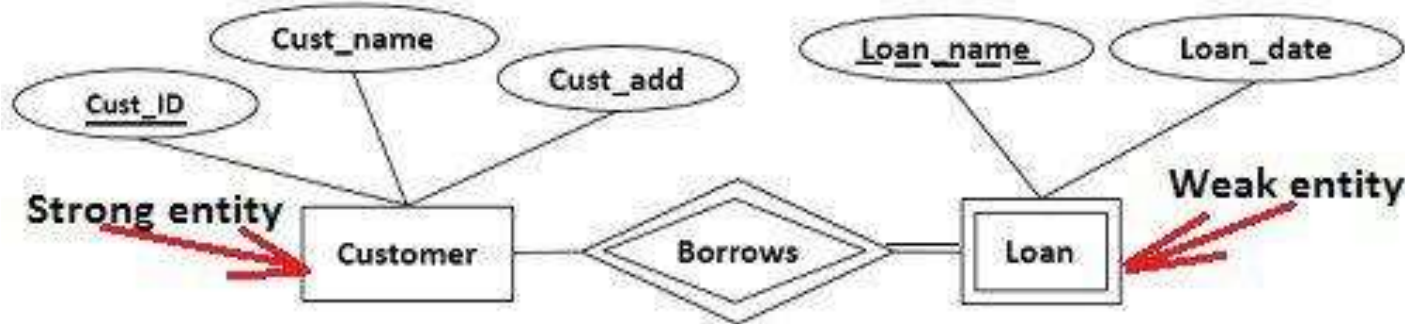


Entity



Entity categories

- **strong entity** can be defined solely by its own attributes.
- **weak entity** cannot be defined solely by its own attributes, An entity set that do not have sufficient attributes to form a primary key.





Entity keys

- Candidate Key
- Primary Key
- Foreign Key
- Composite Key
- Partial Key
- Alternate key
- Super Key



Primary Key

Identifier used to uniquely identify one particular instance of an entity.

- Can be one or more attributes.
- Must be unique .
- Value should not change over time.
- Must always have a value .



Entity keys

- **Candidate Key :**

when multiple possible identifiers exist, each is a candidate key.

- **Foreign Keys :**

Foreign keys reference a related table through the primary key of that related table.

- **Referential Integrity Constraint:**

For every value of a foreign key there is a primary key with that value in the referenced table e.g. if student name is to be used in a dormitory table then that name must exist in the student table.



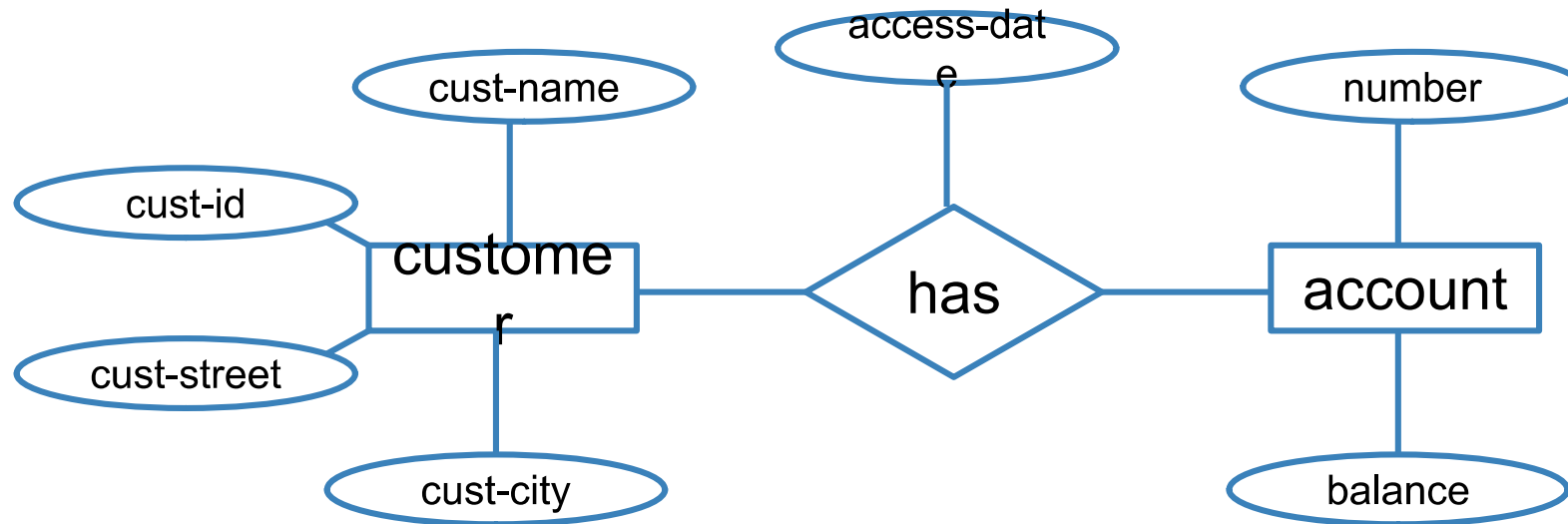
Attribute

- A property or characteristic of an entity. Often shown as an oval or circle.
- **Attribute categories:**
 - ☐ Simple (Scalars) Attribute
 - ☐ Composite Attribute
 - ☐ Derived Attribute
 - ☐ Multi-valued Attribute
 - ☐ Complex Attribute



Simple Attribute

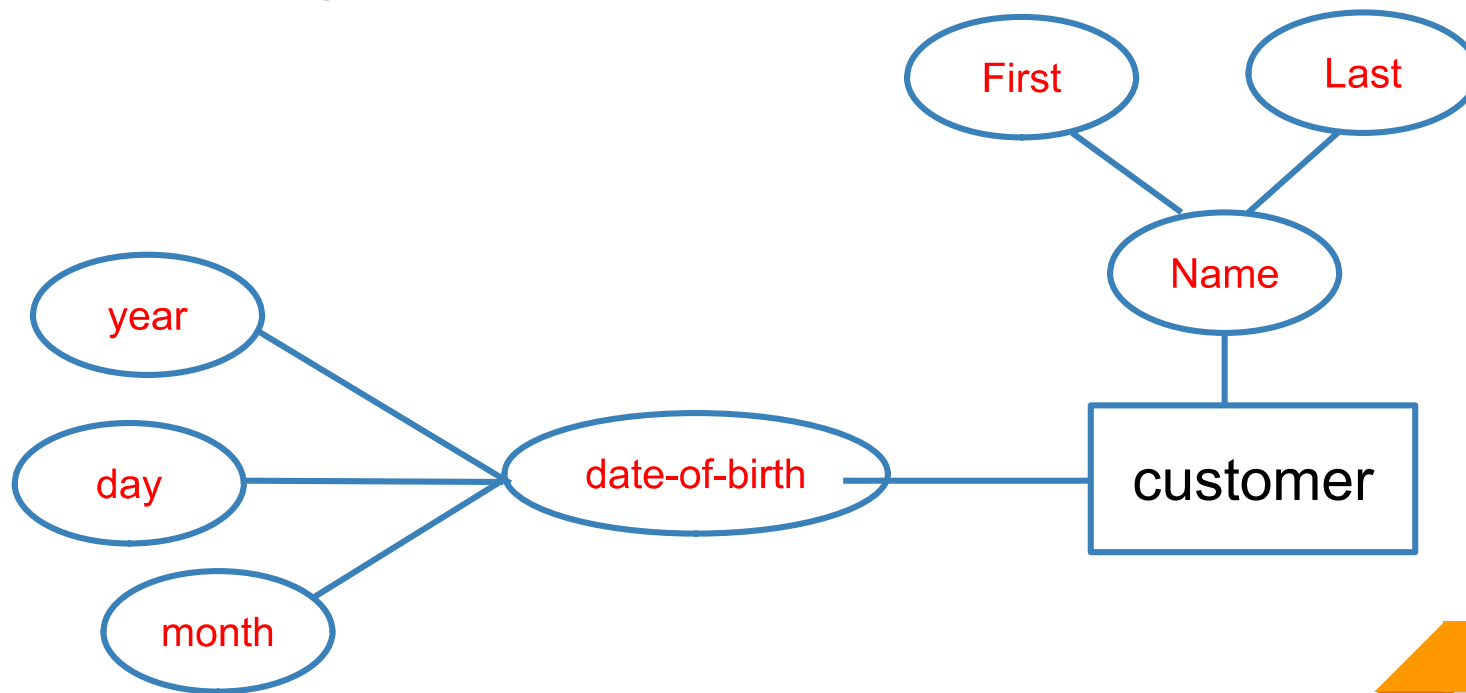
- Means the attribute value is atomic and can't be further divided, such as a phone number.





Composite Attribute

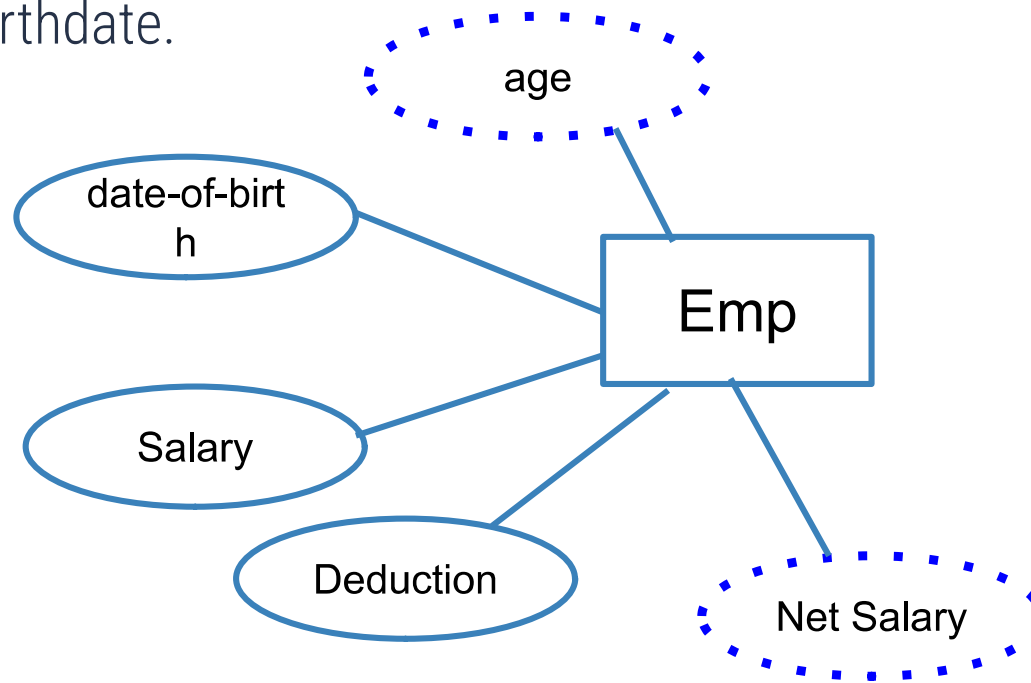
- Sub-attributes spring from an attribute





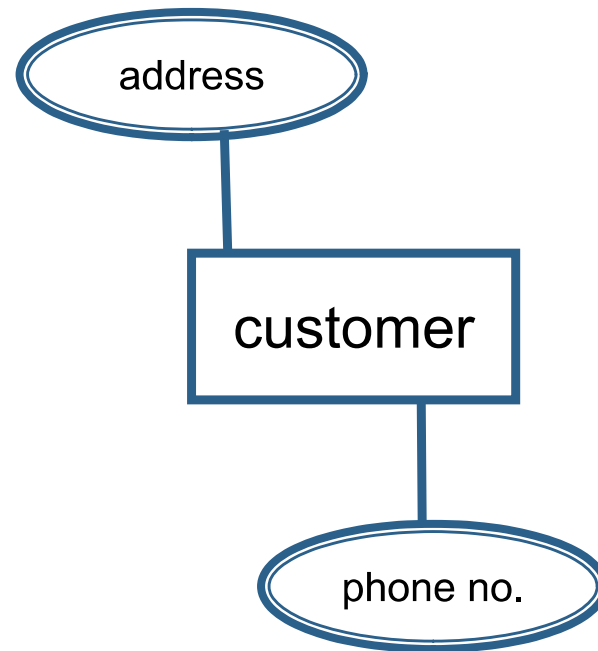
Derived Attribute

- Attributed is calculated or otherwise derived from another attribute, such as age from a birthdate.



Multi-valued

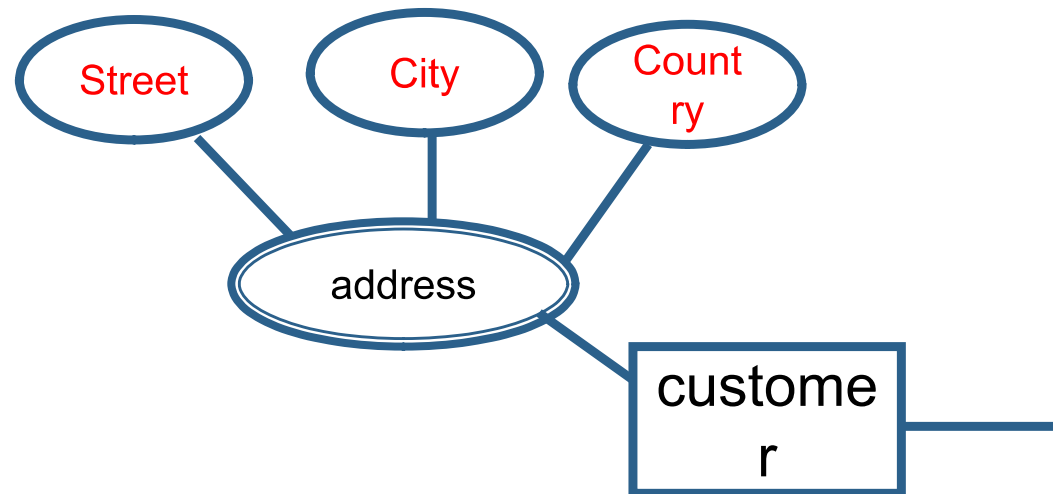
- More than one attribute value is denoted, such as multiple phone numbers for a person.





Complex Attribute

- multi-valued + Composite





Relationship

- A Relationship is an association among several entities.
- A relationship may also have attributes
- For example, consider the entity sets customer and loan and the relationship set borrower. We could associate the attribute date-issued to that relationship to specify the date when the loan was issued.

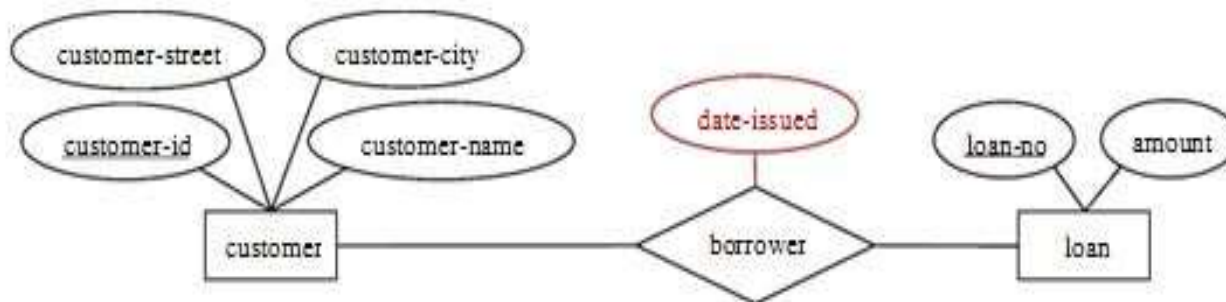


Figure: Descriptive attribute date-issued.



Relation Properties

Relation has three Properties:

- ☐ Degree of Relationships
- ☐ Cardinality Constraint
- ☐ Participation Constraint

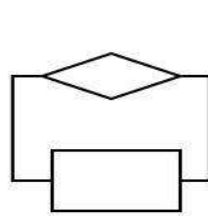


Degree of Relationships

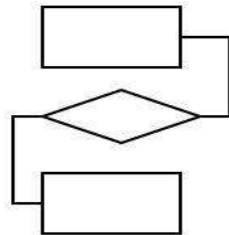
- number of entity types that participate in a relationship

Three cases:

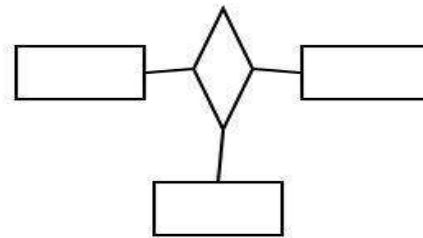
- ☐ **Unary:** between two instances of one entity type
- ☐ **Binary:** between the instances of two entity types
- ☐ **Ternary:** among the instances of three entity types



Unary



Binary

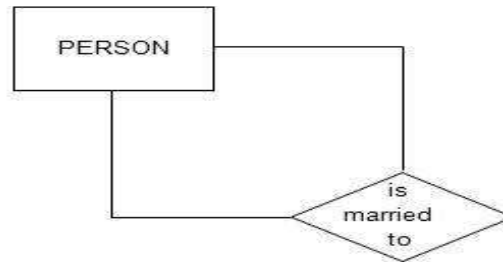


Ternary



Degree of Relationships

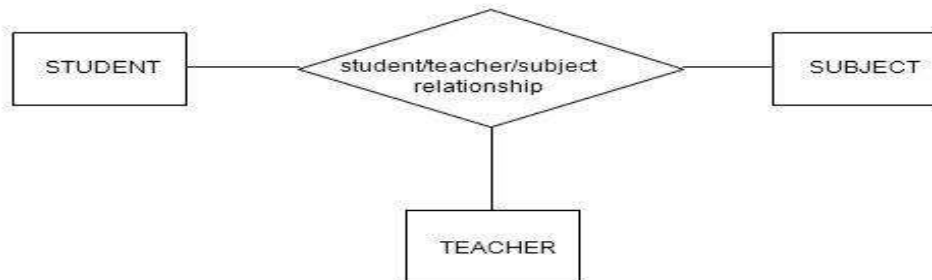
Unary



Binary



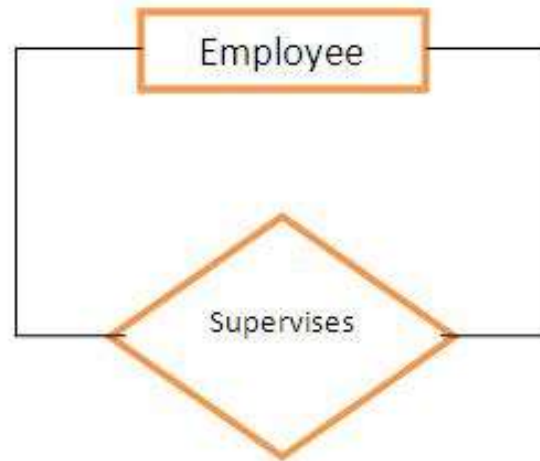
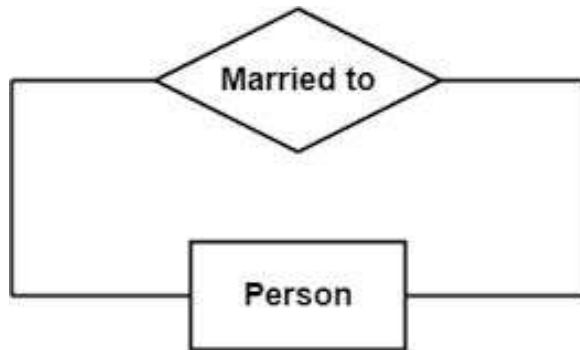
Ternary





Recursive Relationship (Unary)

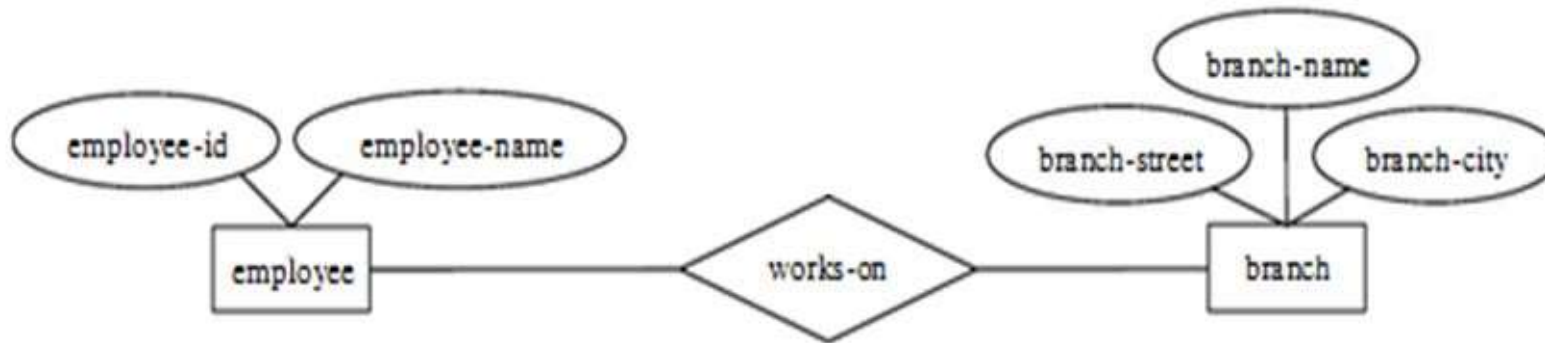
- ❑ **Recursive Relationships** - A relationship in which the same entity participates more than once.





Binary Relationship

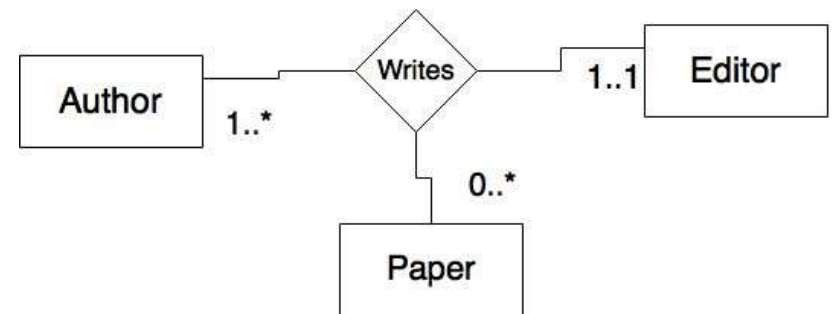
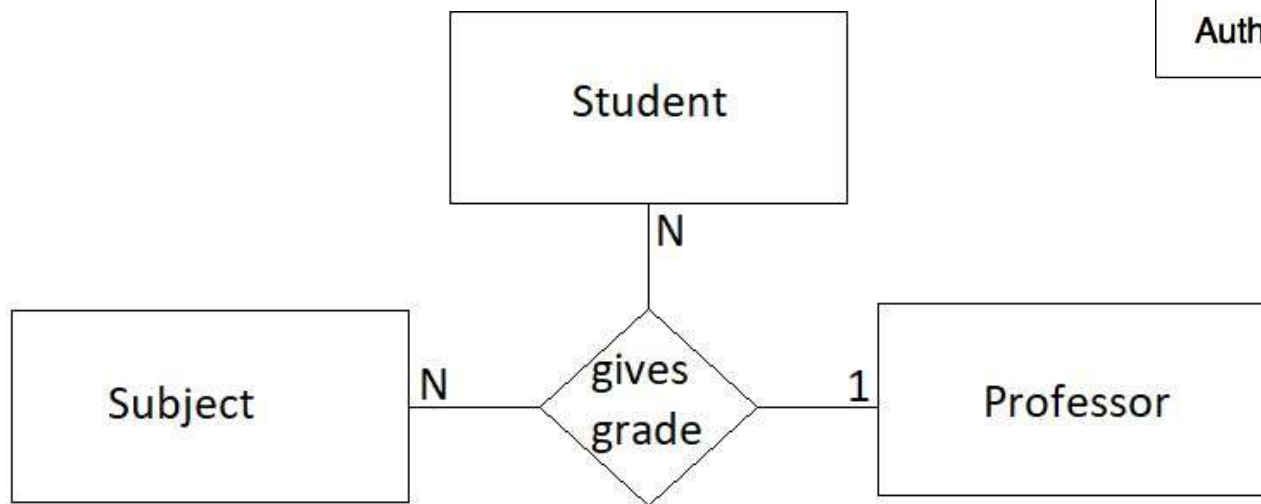
- A **binary relationship** set is of degree 2.





Ternary Relationship

- Ternary relationship set is of degree 3.





Cardinality

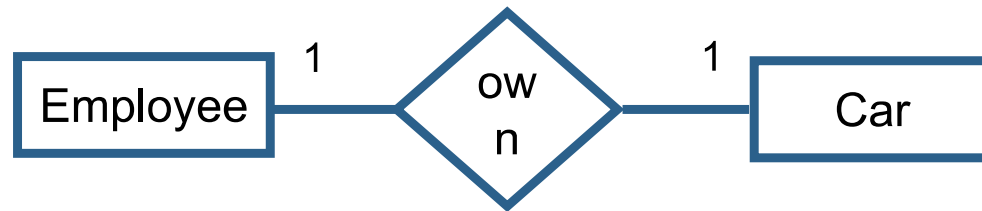
How many instances of one entity will or must be connected to a single instance from the other entities.

- One-One Relationship
- One-Many Relationship
- Many- Many Relationship

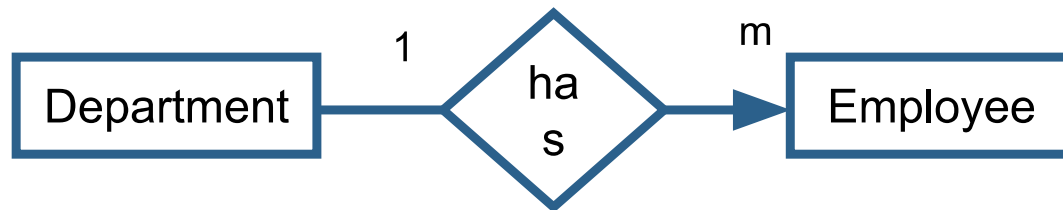


Mapping Cardinalities

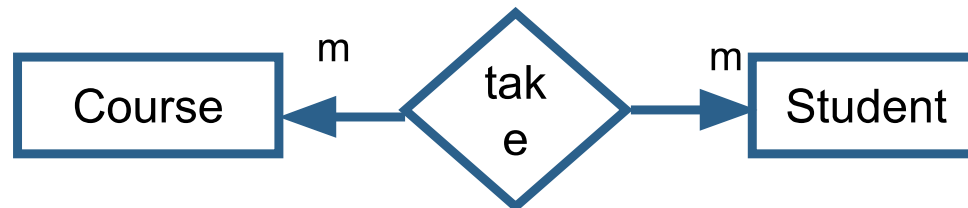
■ One-to-One



■ One-to-Many



■ Many-to-Many





PARTICIPATION CONSTRAINT

- An employee **MUST** work for a department

An employee entity can exist only if it participates in a WORKS_FOR relationship instance

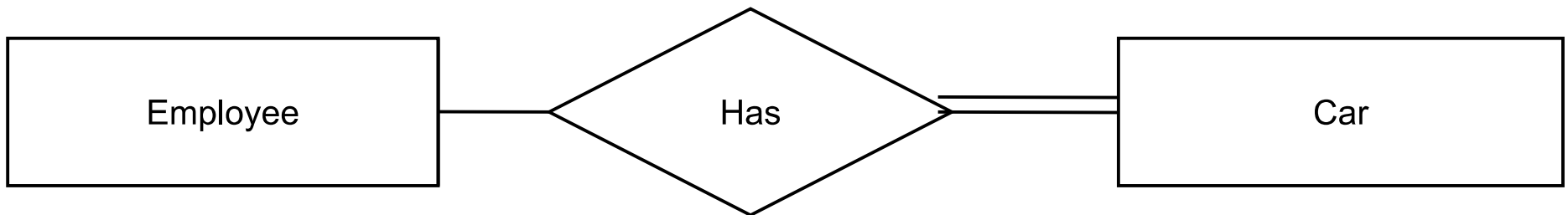
So this participation is **TOTAL**

- Only some employees manage departments

The participation is **PARTIAL**



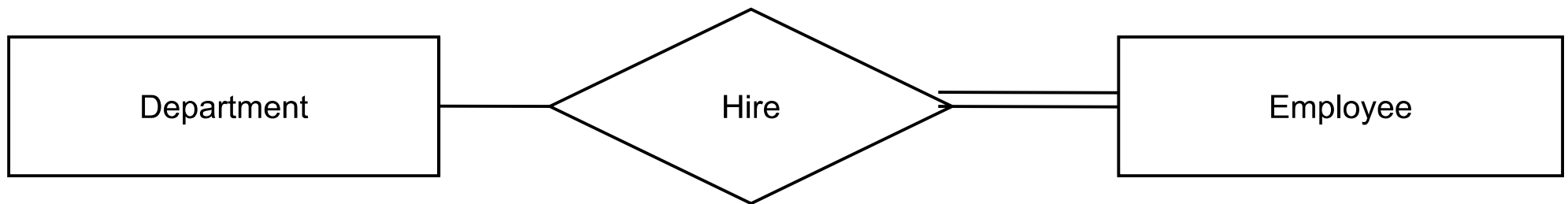
PARTICIPATION CONSTRAINT



- An Employee **may** have a car.
- A Car **must** be assigned to particular employee



PARTICIPATION CONSTRAINT



- A department **may** hire many employees (Zero or more)

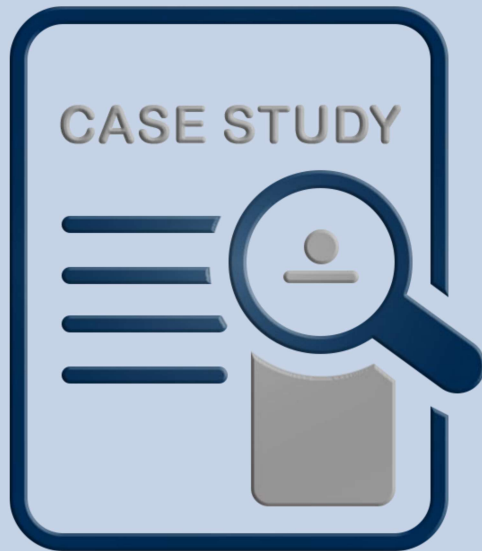
- An employee **must** be employed by a department

(Department membership is **Optional**, Employee membership is **Mandatory**)



Summary of notation for ERD

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1 : N for $E_1:E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

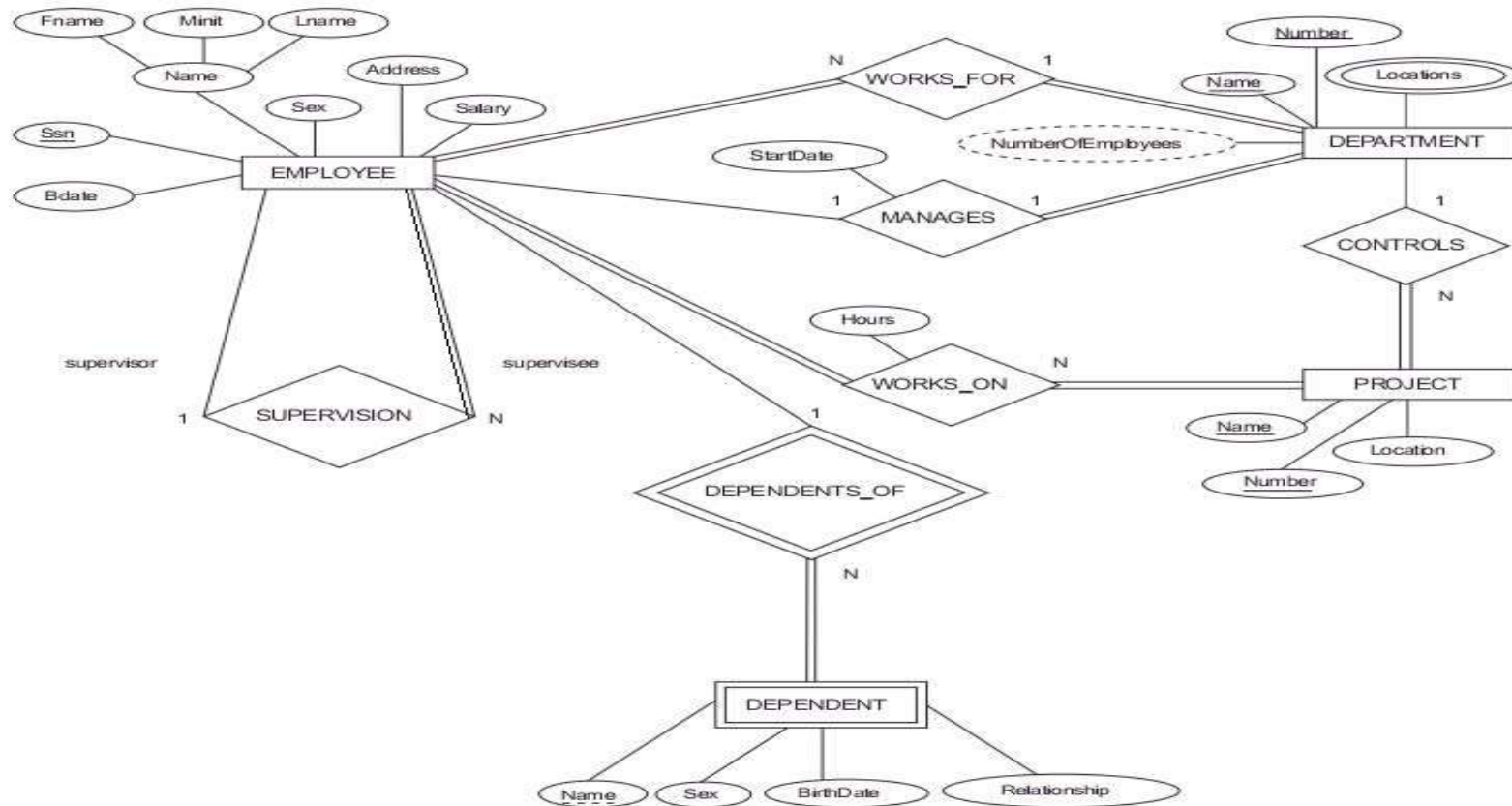


Case Study

Company



Result





THANKS!

Any questions?